Kabul University
Faculty of Computer Science

Software Engineering Department

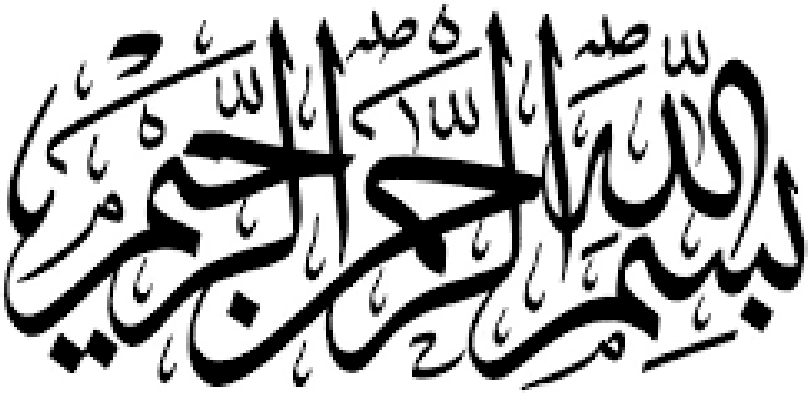
Nokta Academy Management System

A Project Submitted in Partial Fulfillment
of the Requirements for the Degree of
Bachelor of Science in Software Engineering

Mohammad Maiwand Osmani
Mohammad khalid Sahil

Supervisor: Associate Professor Abdul Wahid Samadzai

June 5, 2026


بسم الله الرحمن الرحيم

ABSTRACT

Nokta Academy currently faces academic and administrative inefficiencies caused by manual records, disconnected spreadsheets, delayed reporting, limited communication, and weak financial transparency. This monograph presents the Nokta Academy Management System (NAMS), an integrated web and mobile-oriented platform designed to automate student enrollment, class and subject management, attendance, grading, notifications, fee payments, teacher payroll, tax calculation, reporting, and role-based access. The system uses a modern full-stack architecture built with Node.js, Express.js, MongoDB, React.js, and mobile-ready application components. It also prepares a structured foundation for AI-based learning analytics and student performance monitoring. The methodology combines Agile Scrum with structured SDLC phases. Requirements were gathered through questionnaire-based needs assessment, workflow analysis, role analysis, and proposal review. From a population of 100 stakeholders, 75 valid responses were selected, and 64 respondents approved the project, producing an 85.33% acceptance rate. The system was designed through architecture modeling, use case modeling, class modeling, database ERD, functional and non-functional requirement specification, and security design. The results show that NAMS can reduce manual work, improve data accuracy, centralize communication, strengthen financial accountability, and support evidence-based decision-making. The project contributes a context-aware digital transformation model for private academies in Afghanistan while following software engineering standards, APA citation practices, copyright awareness, and Mendeley-compatible reference management.

TABLE OF CONTENTS

ABSTRACT 1

TABLE OF CONTENTS 4

LIST OF FIGURES 6

LIST OF TABLES 6

LIST OF ABBREVIATIONS 7

CHAPTER 1: INTRODUCTION 8

1.1 Background 8

1.2 Problem Statement 9

1.3 Objectives 9

1.3.1 General Objective 9

1.3.2 Specific Objectives 10

1.4 Scope and Limitations 10

1.4.1 Scope (Inclusions) 10

1.4.2 Limitations (Exclusions) 11

1.5 Significance of the Project 11

1.6 Organization of the Report 11

CHAPTER 2: LITERATURE REVIEW 12

2.1 Concepts and Definitions 12

2.2 Related Work/Existing Systems 13

2.3 Recent Trends and Technologies 14

2.4 Research Gap Synthesis 15

2.5 Copyright and Plagiarism Considerations in Literature Use 16

2.6 Learning Analytics and the AI Direction of the Proposal 16

2.7 Financial Transparency and Educational Administration 17

2.8 Security, Privacy, and Role-Based Access Control 18

2.9 User Acceptance, UX, and Local Adoption 19

2.10 Chapter Summary 20
CHAPTER 3: Methodology 21
3.1 Requirements Elicitation Methodology 21
3.2 Functional Requirements 22
3.3 Non-Functional Requirements 23
3.4 Use Case Model 24
3.5 Data Requirements 25
3.6 Research Methodology: Why Mixed-Methods Was Used 26
3.7 Design Methodology: Why Hybrid Agile-SDLC Was Used 27
3.8 Requirements Traceability to Research Questions 28
3.9 Requirement Validation and Prioritization 29
3.10 Ethical and Data Protection Requirements 30
3.11 Design Methodology 31
3.12 Architectural Design 32
3.13 Database Design 33
3.14 Class Diagram 35
3.15 Detailed Design 36
UI/UX Design 36
Algorithm Design 36
3.16 Security Design 36
3.17 Testing Strategy 37
CHAPTER 4: DISCUSSION 38
CHAPTER 5: CONCLUSION 41
APPENDIX A: TECHNICAL ARTIFACTS 41
REFERENCES 42

LIST OF FIGURES

Figure	Title
Figure 3.1	Use Case Diagram for Nokta Academy Management System
Figure 4.1	Layered Architecture of NAMS
Figure 4.2	Entity-Relationship Diagram
Figure 4.3	Class Diagram
Figure 4.4	Hybrid Agile-SDLC Workflow
Figure 5.1	Project Acceptance Result
Figure 5.2	Feature Priority Chart
Figure 5.3	User Evaluation Indicators

LIST OF TABLES

Table	Title
Table 1.1	Research Questions and Objective Alignment
Table 2.1	Comparison of Existing Systems
Table 2.2	Literature Gap Synthesis
Table 3.1	Functional Requirements
Table 3.2	Non-Functional Requirements
Table 4.1	Technology Stack and Justification
Table 4.2	Core Relational/Document Schema
Table 5.1	Research Question Answer Matrix

LIST OF ABBREVIATIONS

Abbreviation	Meaning
AI	Artificial Intelligence
API	Application Programming Interface
CRUD	Create, Read, Update, Delete
DBMS	Database Management System
ERD	Entity Relationship Diagram
FR	Functional Requirement
IEEE	Institute of Electrical and Electronics Engineers
JWT	JSON Web Token
LMS	Learning Management System
NAMS	Nokta Academy Management System
NFR	Non-Functional Requirement
RBAC	Role-Based Access Control
SDLC	System Development Life Cycle
UI	User Interface
UAT	User Acceptance Testing
UX	User Experience

CHAPTER 1: INTRODUCTION

1.1 Background

Digital transformation has become a central strategy for educational institutions that need accurate data, faster administration, transparent finance, and better communication. In many developing contexts, including Afghanistan, academies still depend on paper files, handwritten ledgers, or disconnected spreadsheets for enrollment, scheduling, payment tracking, grades, and attendance. These tools may appear simple at the beginning, but they become unreliable when student numbers, teacher assignments, financial transactions, and reporting obligations increase. Research on learning analytics and educational data mining shows that educational data becomes valuable when it is collected consistently and transformed into actionable information (Aldowah et al., 2019).

Nokta Academy Management System is designed as an integrated web and mobile-oriented platform that centralizes academic, administrative, communication, and financial operations. It supports student registration, class and subject management, attendance, exams, results, notifications, payments, teacher salary, tax calculation, academy financial reports, and dashboards. This integration responds directly to the proposal's main idea: developing a modern online academy platform with Node.js, Express.js, MongoDB, React.js, Flutter-oriented access, and AI-ready analytics.

The system also reflects the global movement toward smart educational systems. Smart educational platforms are not limited to online content delivery; they involve data-driven administration, user-centered design, real-time communication, and analytics-supported decision-making (Al-Samarraie et a2020). In NAMS, dashboard analytics, financial reporting, and student performance modules provide a foundation for future AI features such as risk detection and personalized recommendations.

From a software engineering perspective, the project is important because it translates a proposal into a working artifact. The project uses an iterative structure, modular APIs, role-based access, MongoDB schemas, and React-based interfaces. Software engineering literature emphasizes that successful systems should be developed from clear requirements, designed with maintainable architecture, and tested against expected behavior (Pressman et al 2020).

The local importance of NAMS is also visible in its financial workflows. Many academy systems handle grades and attendance, but do not calculate teacher commission salary,

government salary tax, student fee balances, and academy financial reports in an integrated way. NAMS connects student payments with teacher payroll and reporting. This makes the project more than an ordinary record-keeping system; it becomes a management decision-support platform for academy operations.

1.2 Problem Statement

Nokta Academy experiences several academic and administrative inefficiencies due to the absence of a centralized digital management system. Essential operations such as student enrollment, attendance tracking, grading, timetable management, salary calculation, fee collection, and reporting are difficult to control when they are handled through manual documents or separate spreadsheets. This fragmentation creates data inconsistency, duplicate effort, delayed reporting, and weak transparency.

The second major problem is communication. Students, teachers, parents, and administrators need a common platform for schedules, results, announcements, messages, and academic feedback. Without such a platform, updates may be delayed or lost. Educational technology literature identifies communication and timely feedback as important components of effective digital learning environments (DeLone et al., 2003).

The third problem is the lack of data-driven academic monitoring. The proposal highlights AI-powered analytics and student performance prediction. Although the current implementation focuses mainly on management and reporting, its data model creates the foundation for later AI modules. Learning analytics research shows that student records, performance data, attendance, and interaction data can support early risk detection and personalized learning support when collected systematically (Delen, et al 2020).

The gap, therefore, is not only the absence of a software product. The real gap is the absence of an integrated workflow where academic data, financial data, communication, and analytics are stored and processed together. NAMS is proposed to close this gap by connecting stakeholders, modules, and reports in one controlled system.

1.3 Objectives

1.3.1 General Objective

To design, develop, and implement an integrated web-mobile academy management system for Nokta Academy using modern JavaScript-based technologies that automate academic,

administrative, communication, and financial operations while preparing a foundation for AI-based analytics and personalized learning support.

1.3.2 Specific Objectives

Table 1.1: Research Questions and Objective Alignment

Research question	Aligned objective	Evidence in project/report
How can an integrated online academy management system enhance operational efficiency and personalized learning?	Develop a centralized platform that integrates academic, administrative, financial, and analytics-ready workflows.	Architecture, dashboard, modules, data model, and evaluation.
What current inefficiencies can be addressed through digitization?	Analyze manual and fragmented workflows through questionnaire and workflow analysis.	Problem statement, requirements analysis, and survey results.
How can AI-driven analytics improve student learning outcomes and monitoring?	Prepare structured academic and performance data for future analytics and risk prediction.	Student, attendance, result, and dashboard data model.
What are best practices for communication and financial transparency?	Implement messaging, notifications, payments, payroll, salary tax, and reports.	Messages, notifications, payments, payroll, finance report.
How can Node.js, MongoDB, React, and Flutter support scalable educational platforms?	Design a layered architecture using Node/Express API, MongoDB, React frontend, and mobile-ready access.	Technology stack table and architecture diagram.

Every research question from the proposal is mapped to a matching objective and project evidence.

1.4 Scope and Limitations

1.4.1 Scope (Inclusions)

The scope includes a web-based academy management system with API support and mobile-ready design. Included features are authentication, user roles, student registration, teacher management, class and subject management, timetable, attendance, exams, results, notifications, messages, fee payments, expenses, payroll, salary tax, financial reports, dashboards, and audit-related visibility. Target users include super administrators,

administrators, teachers, students, parents, owners, and branch managers. Data includes student profiles, academic records, attendance, fees, salary records, tax values, messages, and notifications.

1.4.2 Limitations (Exclusions)

The system is designed for Nokta Academy and similar academies, but its evaluation was limited to a 100-person population with 75 sampled respondents. The AI analytics component is prepared conceptually and structurally, but advanced machine learning prediction is a future enhancement rather than the central evaluated component. Production deployment, national-level integration, and long-term multi-academy performance benchmarking are outside the current project scope.

1.5 Significance of the Project

The project is practically significant because it reduces manual work, improves data consistency, and creates transparent financial workflows. It is academically significant because it applies software engineering, requirements engineering, database design, user acceptance, and learning analytics literature to a real project. It is socially significant because it supports digital capacity building in Afghanistan's education sector. It is personally significant for the developers because it demonstrates end-to-end software engineering skills from proposal to implementation and evaluation.

1.6 Organization of the Report

Chapter 1 introduces the background, problem, objectives, scope, limitations, and significance. Chapter 2 reviews concepts, existing systems, recent technologies, and research gaps. Chapter 3 presents requirements analysis, including elicitation methodology, functional requirements, non-functional requirements, use cases, and data requirements. Chapter 4 explains system design, including methodology, architecture, database design, UI/UX, algorithm design, security, and testing strategy. Chapter 5 presents results and discussion. Chapter 6 concludes the report and provides recommendations. Appendices contain technical artifacts, references, citation map, and code-related summaries.

CHAPTER 2: LITERATURE REVIEW

2.1 Concepts and Definitions

This literature review focuses on concepts that directly support the proposal and implementation of NAMS. An academy management system is treated as an educational information system, not merely as a website. It integrates operational data, academic records, communication, finance, and reporting. The DeLone and McLean model is relevant because it evaluates information systems through system quality, information quality, service quality, use, user satisfaction, and net benefits (DeLone et al 2003).

Learning analytics and educational data mining are relevant because the proposal includes AI-based analytics and performance monitoring. Aldowah et al. (2019) and Romero and Ventura (2020) show that educational data can support decision-making, risk detection, and learning improvement when it is collected and analyzed systematically. For NAMS, this means that student attendance, grades, payments, and timetable data should be stored in a way that supports future analysis.

Requirements engineering is the process of discovering, documenting, validating, and managing stakeholder needs. ISO/IEC/IEEE 29148:2018 is used as a standard reference because it emphasizes clear, testable, necessary, and traceable requirements. In NAMS, requirements are traced from the proposal, questionnaire, and workflow problems to implemented modules such as Students, Payments, Payroll, Reports, Dashboard, and Role Management (International Organization for Standardization et al., 2018).

Software quality is evaluated using ISO/IEC 25010:2011, which includes functional suitability, usability, security, performance efficiency, reliability, maintainability, compatibility, and portability. This standard is important because NAMS handles sensitive academic and financial data, so quality must include security and maintainability in addition to visible features (International Organization for Standardization & International Electrotechnical Commission, 2011).

The Technology Acceptance Model is used because the system must be accepted by real users, not only completed technically. Davis (1989) explains that perceived usefulness and perceived ease of use influence acceptance. This is why the questionnaire measures ease of use, system necessity, speed, interface simplicity, mobile access, and future willingness to use the system.

2.2 Related Work/Existing Systems

Existing educational platforms provide useful lessons but do not fully satisfy the localized needs of Nokta Academy. Learning management systems such as Moodle and Canvas focus strongly on course content and learning delivery, while student information systems such as PowerSchool and openSIS focus on institutional records. Google Classroom is simple and widely used for class communication, but it does not provide integrated academy finance, salary tax, local payroll, or custom role-based workflows. NAMS fills this gap by combining academic management, communication, financial transparency, payroll, and reporting in one context-aware platform.

Table 2.1: Comparison of Existing Systems

System	Features	Technology/Type	Pros	Cons	Gap addressed by NAMS
Moodle	Courses, assignments, quizzes, LMS tools	PHP-based LMS	Strong learning content management	Not focused on local academy finance and payroll	Adds finance, payroll, tax, and local workflows
Google Classroom	Class posts, assignments, file sharing	Cloud education platform	Simple and accessible	Limited institutional management and reporting	Adds student records, payments, payroll, dashboards
Canvas LMS	Course management, grading, integrations	Cloud/enterprise LMS	Modern UI and integrations	Complex and not localized for small academies	Provides localized academy workflow
openSIS	Student information, attendance, grades	Student information system	Good SIS foundation	Limited custom payroll and tax workflows	Adds integrated salary tax and financial reports

Fedena	School ERP, attendance, finance	ERP-style system	Broad school operations	May require customization and licensing	Custom-built for Nokta Academy needs
NAMS	Academic, finance, payroll, reports, roles, dashboards	Node.js, Express, MongoDB, React, Flutter-ready	Localized, modular, integrated	AI prediction still future enhancement	Directly addresses proposal objectives

The comparison shows why a custom system is justified for Nokta Academy.

The table above is directly connected to the project gap. Existing systems either focus on course delivery, student information, or general school ERP functions, but NAMS combines local academy operations, fee tracking, teacher commission salary, tax reporting, and role-based dashboards. This relationship between the comparison table and the project design is important because it prevents the literature review from becoming a list of unrelated systems.

2.3 Recent Trends and Technologies

A major trend in educational technology is the movement from isolated digital tools to integrated smart education systems. Such systems combine data collection, dashboards, analytics, communication, and personalization. Learning analytics literature shows that institutions can use student data for monitoring progress and identifying academic risk (Ifenthaler et al 2020). The proposal's AI direction is therefore consistent with recent educational technology trends.

The MERN-style technology stack is also consistent with modern scalable web development. Node.js and Express.js support asynchronous API development, MongoDB supports flexible document storage, and React supports interactive user interfaces. These tools match the proposal's objective of building a scalable, maintainable, and responsive academy platform. From an architecture viewpoint, separating the client, API, data, and security layers improves maintainability and modifiability (Bass et al., 2012).

Agile and SDLC hybrid development is another relevant trend. Agile supports feedback, iteration, and adaptive planning, while SDLC preserves structured documentation, requirement analysis, design, testing, deployment, and maintenance. Dikert et al. (2016) show that agile transformations require clear coordination and disciplined practices. For a two-person student

team, combining Agile Scrum practices with SDLC phases provides both flexibility and academic documentation discipline.

In NAMS, these technologies are not selected only because they are popular. Node.js and Express are suitable because the system requires many API endpoints and middleware for authentication, validation, permissions, and reporting. MongoDB is suitable because academy data contains structured relationships but also flexible fields such as permissions, messages, notifications, and audit histories. React is suitable because administrative dashboards require dynamic components, charts, forms, and role-specific navigation. Flutter is included in the proposal as the mobile application direction because students and teachers need schedule, notification, and result access outside desktop environments.

2.4 Research Gap Synthesis

The literature and related systems show four main gaps addressed by NAMS. First, many systems focus either on learning management or student records, but not on integrated academic-financial workflows. Second, existing tools are often not localized for Afghan academy operations, payroll expectations, multilingual needs, and local financial reporting. Third, many small academies lack real-time dashboards and role-based visibility. Fourth, AI and learning analytics require structured data, but manual systems do not collect data consistently enough for future prediction and recommendation.

Table 2.2: Literature Gap Synthesis

Identified gap	Evidence from literature/proposal	NAMS response
Fragmented academic and financial workflows	Educational systems often separate learning, administration, and finance	Integrated students, payments, payroll, finance, and reports
Weak data foundation for AI analytics	Learning analytics requires structured and reliable data	Stores attendance, grades, payments, and student relationships
Limited local financial transparency	Proposal highlights financial transparency and reporting	Adds salary, tax, net payment, financial report
Weak user acceptance when systems are hard to use	TAM emphasizes usefulness and ease of use	Clickable dashboard, multilingual UI, light/dark mode

Lack of role-based visibility	Sensitive academic and financial data require controlled access	JWT, RBAC, route permissions, protected routes

This table shows how the literature review directly supports the project rationale.

The synthesis confirms that NAMS is not a duplicate of an existing platform. Its contribution is contextual integration. It connects the academic, financial, administrative, and communication workflows of Nokta Academy while preparing a foundation for future learning analytics. This makes the project consistent with both the original proposal and current educational technology research.

2.5 Copyright and Plagiarism Considerations in Literature Use

Academic writing must distinguish between source-supported ideas and copied text. This monograph uses sources to support arguments, standards, and methodological choices, but the wording is original and project-specific. APA 7th edition style is used for in-text citations and the reference list (American Psychological Association, 2020). A Mendeley-compatible RIS file is provided so that all sources can be imported and managed in Word through Mendeley Cite.

2.6 Learning Analytics and the AI Direction of the Proposal

The original proposal states that NAMS should support AI-powered analytics, performance prediction, and personalized learning recommendations. The current implemented project mainly provides the management platform and structured data foundation, but the literature review must still explain why the AI direction is academically valid. Learning analytics is concerned with collecting, measuring, analyzing, and reporting learner data so that institutions can understand and improve learning processes. Siemens (2013) argues that learning analytics emerged as a discipline because educational institutions increasingly need evidence-based insight from digital traces. In the Nokta context, such digital traces include attendance, results, timetable participation, teacher assignment, and fee-related continuity.

Educational data mining research supports the same direction from a more computational perspective. Romero and Ventura (2020) describe updated methods for discovering patterns in educational data, while Aldowah et al. (2019) synthesize how educational data mining and learning analytics can support higher education. For NAMS, these studies justify storing student records in a structured and consistent way. Even if the final AI model is a future

enhancement, the system must be designed now so that the data needed for prediction will exist later.

This connection is important because the project should not claim that AI appears automatically. AI requires clean, relevant, and ethically collected data. NAMS therefore contributes to the AI objective first by creating reliable data pipelines: student profiles, attendance, exam results, teacher assignments, and payment history. Future AI models can then use this foundation to predict academic risk, identify irregular attendance patterns, or recommend support actions.

Table 2.3: Learning Analytics Data Requirements and NAMS Support

AI/analytics need	Required data in NAMS	Current project support	Future AI use
Performance prediction	Exam scores, attendance, class, subject, teacher	Results, attendance, timetable modules	Risk detection for weak students
Personalized recommendation	Subject performance, learning history, attendance trend	Student and result records	Suggested learning resources
Administrative monitoring	Enrollment, payments, attendance, reports	Dashboard and finance reports	Operational forecasting
Student retention support	Attendance drop, low scores, payment interruption	Structured student records	Early warning model

This table follows the AI discussion and shows that the analytics objective depends on data already collected by the system.

The table demonstrates that the AI-related proposal objective is not separate from the management system. Instead, AI-readiness depends on the correctness of the core management modules. If attendance, grades, and student identity are inconsistent, prediction will be unreliable. Therefore, the current implementation is a necessary first phase for the future AI module.

2.7 Financial Transparency and Educational Administration

Financial transparency is one of the strongest practical reasons for developing NAMS. In many academies, fee payments, teacher salaries, expenses, and tax-related values are calculated separately. This separation creates risk: a student may be shown as unpaid in one document and paid in another; a teacher salary may be calculated manually without a clear base; financial

reports may not match actual payment records. Information systems literature emphasizes that high-quality information should be accurate, timely, relevant, and accessible to authorized users (DeLone ,2003).

The implemented project responds to this problem by linking student payments to the finance module, teacher salary, payroll records, and academy financial reports. This is especially important because the user requested a scenario with 100 students, 10 teachers, 10 students per teacher, 1,000 AF per student, and random teacher salary percentages between 20% and 40%. That scenario is not only test data; it demonstrates that the financial model can be verified mathematically.

Table 2.4: Financial Transparency Problems and System Responses

Financial process	Manual risk	NAMS design response
Student fee payment	Wrong remaining balance or missing receipt	Payment record with student, class, subject, teacher, amount
Teacher salary	Manual percentage calculation error	Commission based on assigned student payments
Salary tax	Incorrect deduction or unclear net salary	Gross, tax, and net salary stored in SalaryRecord
Academy report	Delayed or inconsistent finance summary	Academy Financial Report and dashboard aggregation
Auditability	No trace from payment to report	Linked records and immutable payment reference

The table directly links literature-supported information quality concerns with project finance features.

After reviewing the table, the relationship between financial transparency and database design becomes clear. The system cannot produce reliable reports if payment, student, teacher, and salary records are not connected. This is why the ERD and class diagram in the design chapter are essential parts of the monograph, not decorative images.

2.8 Security, Privacy, and Role-Based Access Control

NAMS stores sensitive data: student identity, family contact, grades, attendance, payment records, salary values, tax deductions, user accounts, and internal messages. Because of this sensitivity, security must be included in the literature review and not only in implementation. ISO/IEC 25010 includes security as a software quality characteristic, and this supports the project decision to use authentication, authorization, permissions, and protected routes

(International Organization for Standardization & International Electrotechnical Commission, 2011).

Role-based access control is appropriate because different users require different visibility. A student should view personal information and relevant academic records. A teacher should view assigned students and teaching-related records. An administrator should manage records but may not need full system configuration. A super administrator should manage users, roles, permissions, and global reports. This design also supports the principle of least privilege, meaning users should only receive access required for their work.

Table 2.5: Role-Based Visibility Logic

Role	Main permissions	Reason for restriction
Student	View own results, payments, timetable, messages	Protect other students and financial records
Teacher	View assigned students, attendance, results, salary summary	Limit access to assigned academic scope
Admin	Manage students, teachers, payments, reports	Support operations without unrestricted system control
Parent	View child-related information	Protect unrelated academy records
Super admin	Manage roles, permissions, audit, all modules	System ownership and governance

This table introduces the same security logic that appears later in the Use Case and system design sections.

The table shows that security is not a technical afterthought. It is part of the project's academic and ethical responsibility. Since the system handles minors' records and financial data, role-based access supports both privacy and institutional accountability.

2.9 User Acceptance, UX, and Local Adoption

Technology adoption in an academy depends on whether users believe the system helps them and whether they can use it without excessive effort. Davis (1989) explains this through perceived usefulness and perceived ease of use. Venkatesh et al. (2003) extend the idea by discussing performance expectancy, effort expectancy, social influence, and facilitating conditions. These concepts are important in Afghanistan's academy context because users may have different digital skill levels and may use the system under weak internet conditions.

The questionnaire therefore includes questions about ease of use, speed, simple design, multilingual support, mobile need, and future willingness to use the system. UX decisions in the project were also based on adoption concerns. For example, dashboard cards were made clickable because users should not search through menus when a statistic already represents a module. Light/dark mode readability was improved because information must remain visible in both themes.

Table 2.6: User Acceptance Concerns and UI/UX Responses

UX concern	Questionnaire relation	Design response
Simple navigation	Design simplicity question	Clickable dashboard cards
Speed with weak internet	Speed requirement question	API caching and focused queries
Multilingual use	Language requirement question	English, Dari/Farsi, Pashto labels
Mobile need	Mobile application question	Mobile-ready design and Flutter structure
Trust in financial data	Report and fee questions	Payment summaries and financial reports

The table connects user acceptance literature to actual project interface decisions.

The table reinforces that UX design is not only about colors or layout. It is part of adoption strategy. A system that users cannot understand will not improve academy management, even if the backend is technically correct.

2.10 Chapter Summary

The literature review supports the project from several directions. Educational data mining and learning analytics justify the AI-ready data foundation. Information systems success theory explains why system quality, information quality, and user satisfaction matter. Requirements engineering standards justify traceable functional and non-functional requirements. Software architecture and database literature justify layered design and ERD modeling. UX and technology acceptance literature justify the questionnaire and interface decisions. Together, these sources show that NAMS is aligned with the original proposal, current research, and practical academy needs.

CHAPTER 3: Methodology

3.1 Requirements Elicitation Methodology

Requirements were elicited through questionnaire, workflow analysis, role analysis, proposal review, and direct project inspection. This combination was selected because the system has both human-facing and technical requirements. The questionnaire captures what users need and value. Workflow analysis shows how academy operations actually happen. Role analysis clarifies who should access which part of the system. Proposal review ensures that the final monograph remains aligned with the original approved topic.

The research methodology used in this phase is applied mixed-methods. It is applied because the purpose is to solve a real problem in Nokta Academy. It is mixed-methods because numerical questionnaire data is combined with qualitative workflow interpretation. Creswell and Creswell (2018) recommend mixed-methods research when numerical results and contextual explanation are both needed. Here, numerical results show that 85.33% of sampled respondents approve the project, while workflow analysis explains why the system needs modules such as payments, payroll, reports, and permissions.

The questionnaire used 20 questions grouped into general information, current management status, feature requirements, user experience, information management, current problems, and suggestions. Likert-type questions were used for ease of use, satisfaction, design importance, and system necessity. Likert (1932) is relevant because the project needs to measure attitudes, not only factual responses.

Table 3.1: Requirements Elicitation Methodology

Elicitation method	Reason for use	Output produced
Questionnaire	Measures stakeholder needs and acceptance	75 valid responses and feature priorities
Workflow analysis	Shows how academy tasks are performed	Process-based functional requirements
Role analysis	Identifies access and visibility boundaries	RBAC and permission requirements

Proposal review	Keeps project aligned with approved research plan	Research questions and objectives
Project inspection	Connects report content with actual implementation	Evidence-based design and implementation sections

The table is preceded by an explanation of why multiple elicitation methods were needed and followed by a link to the requirements that each method produced.

The table shows that requirements were not gathered from one source only. This is important because relying only on a questionnaire could miss technical constraints, while relying only on code inspection could ignore user expectations. Combining the methods produces a more reliable requirements baseline.

3.2 Functional Requirements

Table 3.2: Functional Requirements

ID	Requirement	Priority	Source	Verification
FR-001	The system shall allow authorized users to create and manage student profiles with class, subject, teacher, and fee information.	High	Proposal + questionnaire	CRUD and integration test
FR-002	The system shall manage teacher profiles, assigned classes, assigned subjects, and salary percentage settings.	High	Workflow analysis	Unit and UI test
FR-003	The system shall manage classes, subjects, and timetable records.	High	Academic workflow	API and UI test
FR-004	The system shall record attendance for students and support attendance reporting.	High	Questionnaire	Functional test
FR-005	The system shall manage exams, grades, and student results.	High	Proposal	Functional test
FR-006	The system shall record student fee payments and update remaining balance.	High	Finance workflow	Payment test
FR-007	The system shall calculate teacher salary based on assigned student payments and commission percentage.	High	Project update requirement	Payroll calculation test

FR-008	The system shall calculate government salary tax and show gross salary, tax amount, and net payable salary.	High	Financial transparency requirement	Tax service test
FR-009	The system shall provide role-based dashboards with clickable links to related modules.	Medium	UX feedback	Navigation test
FR-010	The system shall support notifications, messages, reports, and printable academy financial reports.	Medium	Questionnaire + proposal	UI and report test
FR-011	The system shall store structured academic data for future AI-based performance monitoring.	Medium	Proposal AI objective	Data model inspection
FR-012	The system shall support multilingual labels and user interface content.	Medium	User experience need	Translation validation

The requirements are written as verifiable system statements and are aligned with the approved proposal.

Functional requirements define what the system must do. In NAMS, these requirements are centered on academic administration, financial transparency, communication, reporting, and analytics-readiness. The verification column is included because requirements are only useful if they can be tested. This follows requirements engineering best practice that requirements should be verifiable and traceable (International Organization for Standardization et al., 2018).

3.3 Non-Functional Requirements

Table 3.3: Non-Functional Requirements

ID	Category	Requirement	Verification
NFR-001	Performance	The system should respond within approximately 2 seconds for common dashboard and CRUD operations under normal academy load.	Manual/API performance check
NFR-002	Security	The system shall enforce JWT authentication and role-based authorization for protected modules.	Security access test
NFR-003	Usability	The system shall provide clear navigation, readable light/dark mode, and role-specific dashboards.	User evaluation

NFR-004	Maintainability	The system shall be organized into modular backend routes/services and reusable frontend components.	Code review
NFR-005	Reliability	Financial calculations shall be repeatable and auditable through stored payment and salary records.	Payroll and payment tests
NFR-006	Compatibility	The system shall support web access and mobile-ready design, with Flutter/mobile extension support.	Responsive UI test
NFR-007	Localization	The system shall support multilingual interface needs, especially English, Dari/Farsi, and Pashto.	Translation test
NFR-008	Data integrity	The system shall maintain relationships among student, class, subject, teacher, payment, and salary records.	Database validation

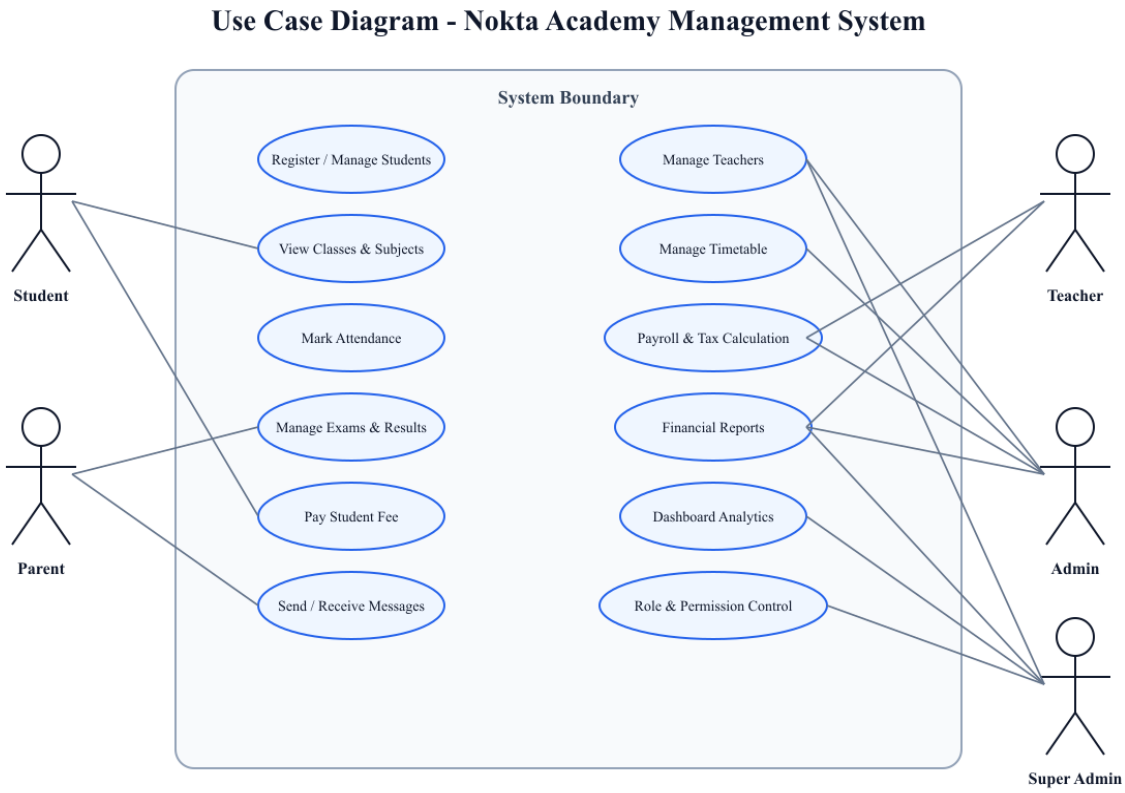
NFR categories are aligned with ISO/IEC 25010 quality characteristics.

Non-functional requirements are especially important because the project handles sensitive data and financial operations. A system may provide features, but if it is insecure, hard to use, slow, or difficult to maintain, it will not succeed. ISO/IEC 25010:2011 provides a quality model that helps classify these concerns into usability, security, maintainability, performance, and reliability (International Organization for Standardization & International Electrotechnical Commission, 2011).

3.4 Use Case Model

The use case model shows how the main actors interact with NAMS. Before the diagram, it is important to clarify that the system is not designed for one user type only. Students need access to schedules, results, and payments. Teachers need access to assigned students, attendance, results, and salary-related information. Administrators need control over academic and financial records. Parents need child-related information. Super administrators need full control over roles, permissions, reports, and system-wide visibility.

Figure 3.1: Use Case Diagram for Nokta Academy Management System



The diagram above visually connects actors with the system functions they use.

After reading the diagram, the connection between roles and requirements becomes clear. The use cases justify why RBAC is necessary: the student actor should not access payroll, the teacher actor should not manage system permissions, and the parent actor should only access child-related information. Therefore, the use case model directly supports both functional and security requirements.

3.5 Data Requirements

The system requires structured data for users, students, teachers, classes, subjects, timetable, attendance, exams, results, payments, salary settings, salary records, expenses, notifications, messages, and audit logs. Data sensitivity is high for student records, payments, salaries, and tax values. Therefore, the system must use authentication, authorization, input validation, and controlled visibility. Data volume in the current seeded scenario includes 100 students, 10 teachers, 10 students per teacher, and 1,000 AF payment per student.

The data requirements also support the AI-related objective in the proposal. AI prediction cannot work without consistent historical data. Student attendance, exam scores, payment patterns, timetable participation, and teacher assignment records can later be used as features

for academic risk detection or personalized recommendations. For the current monograph, AI is treated as a planned extension supported by the system's data foundation.

3.6 Research Methodology: Why Mixed-Methods Was Used

The research side of this project used a mixed-methods approach because the project required both measurable stakeholder evidence and deep understanding of academy workflows. A purely quantitative survey could show that users want digital registration or reports, but it would not explain how payment records should connect with salary or how permissions should differ between students and administrators. A purely qualitative approach could explain workflows but would not produce a measurable acceptance rate. Therefore, the mixed-methods approach was the most suitable research methodology for the proposal and the implemented project (Creswell, 2018).

The quantitative part used questionnaire results from 75 respondents out of a 100-person population. The qualitative part used workflow interpretation and role analysis. This combination made it possible to define requirements that are both evidence-based and technically implementable. For example, the survey showed that reports, payments, attendance, and timetable were important. Workflow analysis then explained how these modules should share data.

Table 3.4: Mixed-Methods Components and Design Impact

Method component	Research purpose	Design impact
Survey/questionnaire	Measures need, priority, and acceptance	Feature ranking, approval rate, UX priorities
Workflow analysis	Understands operational process	Payment-to-payroll flow, attendance flow, result flow
Role analysis	Defines actor responsibilities	RBAC, protected routes, dashboard visibility
Project inspection	Connects monograph to implemented code	ERD, module list, API evidence
Literature review	Provides academic justification	Standards, methods, quality model, adoption theory

The table shows how the research method affected actual design choices, not only data collection.

The table above is important because it proves that methodology was not written after implementation as a formality. Each method produced a specific type of design input. This makes the project more defensible in academic review because the requirements and design choices can be traced back to evidence.

3.7 Design Methodology: Why Hybrid Agile-SDLC Was Used

The design methodology used a hybrid of Agile Scrum and SDLC. Agile Scrum was useful because the project changed through feedback. New needs appeared during implementation, such as improving subject permissions, fixing light/dark mode in reports, adding teacher salary and tax visibility, and making dashboard cards clickable. These changes fit Agile because the team could improve the system incrementally. SDLC was useful because the university project required formal phases: requirement analysis, design, implementation, testing, results, and conclusion.

Using only Agile would not be enough for the monograph because academic assessment requires documented requirements, diagrams, design rationale, and testing strategy. Using only a strict waterfall SDLC would not be practical because user feedback and discovered bugs required iteration. The hybrid approach therefore matches both the academic proposal and the real development process. Literature on agile practice supports iterative improvement, while software engineering texts emphasize structured design and documentation (Dikert et al., 2016).

Table 3.5: Hybrid Agile-SDLC Application

SDLC phase	Agile activity inside phase	Project example
Requirement analysis	Backlog refinement from questionnaire and workflow review	Functional and non-functional requirements
System design	Iterative diagram and model updates	Architecture, ERD, Use Case, Class Diagram
Implementation	Module-by-module sprint work	Students, payments, payroll, dashboard
Testing	Sprint review and bug fixing	Subject access, payroll tax, report display
Deployment planning	Configuration and environment setup	Backend port, frontend API URL, seed data

Maintenance	Change handling after feedback	Currency update, icons, permissions, UI links

This table explains exactly how the chosen methodology was used in the project.

After reviewing the table, it becomes clear why the methodology was not a generic claim. The actual project history shows repeated improvements after testing and feedback. The hybrid method explains this pattern better than either pure Agile or pure waterfall.

3.8 Requirements Traceability to Research Questions

A strong final project should show that each research question has a matching objective, requirement, design decision, and evaluation evidence. This section creates that traceability. It prevents a common monograph weakness: asking broad research questions but never answering them through the design or results.

Table 3.6: Research Question, Objective, Requirement, and Evidence Matrix

Research question	Objective	Requirement/design response	Evaluation evidence
How can an integrated online academy management system enhance operational efficiency and personalized learning?	Develop a centralized web-mobile platform.	Integrated modules and analytics-ready data model.	Dashboard, ERD, acceptance results.
What inefficiencies can be addressed through digitization?	Analyze existing manual gaps.	FRs for registration, attendance, payments, reports.	Problem statement and questionnaire findings.
How can AI-driven analytics improve academic monitoring?	Prepare structured data for future AI.	Attendance, results, timetable, student records.	Learning analytics literature and data requirements.

What are best practices for communication and financial transparency?	Implement messaging, notifications, payments, payroll, reports.	Message, notification, payment, salary, tax modules.	Feature priority and finance design.
How can Node.js, MongoDB, React, and Flutter support maintainability?	Design layered architecture.	Frontend/API/database/mobile-ready layers.	Architecture diagram and technology table.

This matrix ensures that every proposal question is answered by a project objective and implementation evidence.

The matrix shows full alignment between the proposal and the monograph. It also helps the supervisor verify that the final document is not drifting away from the approved topic. Each question has a design response and an evaluation path.

3.9 Requirement Validation and Prioritization

Requirement validation checks whether the identified requirements are correct, complete, consistent, and feasible. In NAMS, validation was performed by comparing four sources: proposal objectives, questionnaire results, academy workflows, and implemented modules. Requirements that appeared in more than one source were treated as higher priority. For example, payments appeared in the proposal, questionnaire, and workflow analysis; therefore, fee payment and finance reporting were high-priority requirements.

Prioritization was necessary because the project had a two-student team and two-semester duration. The system could not implement every possible feature at enterprise scale. Therefore, core academy workflows were prioritized first: authentication, students, teachers, classes, subjects, payments, attendance, results, payroll, reports, and dashboard. AI prediction was treated as a future enhancement because it depends on data accumulated by the operational system.

Table 3.7: Requirement Prioritization

Priority level	Requirement type	Reason
High	Authentication and roles	All other modules depend on secure access

High	Students, teachers, classes, subjects	Core academic data
High	Payments and finance	Major proposal problem and transparency need
High	Payroll and tax	Required by updated project finance workflow
Medium	Messages and notifications	Important for communication but less dependent
Medium	AI analytics	Proposal objective but requires data foundation first
Medium	Advanced exports	Useful but can be expanded after prototype

This table explains why some features were implemented first and others remain future work.

The prioritization table is important because it protects the project from unrealistic scope. It shows that future AI analytics was not ignored; it was logically placed after data collection and management modules.

3.10 Ethical and Data Protection Requirements

Because NAMS stores personal, academic, and financial information, ethical requirements must be included in requirements analysis. Student information, parent contact, grades, attendance status, payments, teacher salary, and tax records should not be visible to unauthorized users. The system must also avoid exposing passwords or sensitive tokens. These requirements are consistent with the security and privacy concerns included in software quality models (International Organization for Standardization & International Electrotechnical Commission, 2011).

The ethical approach in the monograph also includes copyright and plagiarism control. External sources are paraphrased and cited; direct copying is avoided. The reference list follows APA style, and the RIS file enables Mendeley import. Questionnaire results are presented in aggregated form without personal identification.

Table 3.8: Ethical and Data Protection Requirements

Ethical concern	Project response
Student privacy	Role-based visibility and protected routes
Financial confidentiality	Restricted access to payroll, tax, and finance reports
Academic fairness	Structured results and attendance records
Source integrity	APA citations and Mendeley-compatible RIS file

Survey confidentiality	Aggregated results without respondent names

This table connects ethics with both system design and monograph writing.

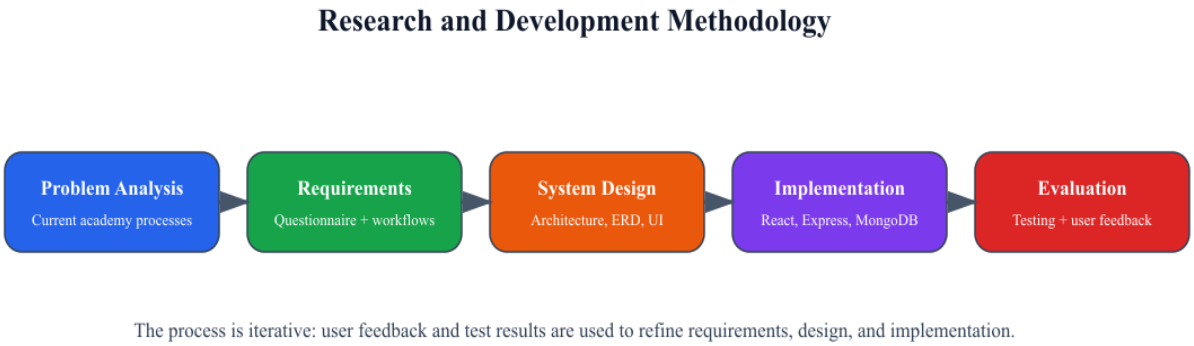
After this ethical analysis, it is clear that the project is not only a technical artifact. It also has responsibility toward students, teachers, parents, and institutional trust.

3.11 Design Methodology

The design methodology combines Agile Scrum and SDLC. Agile Scrum was used to divide work into increments and support feedback. SDLC was used to preserve academic structure: requirement analysis, system design, implementation, testing, deployment planning, and maintenance planning. This hybrid approach matches the proposal and is suitable for a two-member team because it balances flexibility with documentation (Pressman ,2020).

Agile was used in the design process for changing and refining modules. For example, the dashboard navigation, financial report light/dark mode, subjects permission logic, and teacher payroll display were improved after testing. SDLC was used to keep the project report organized into phases and deliverables. This combination is supported by literature on agile practice and structured software development (Dikert et al., 2016).

Figure 3.10: Hybrid Agile-SDLC Workflow



The figure summarizes the development methodology. It is placed after the explanation and followed by the design interpretation below.

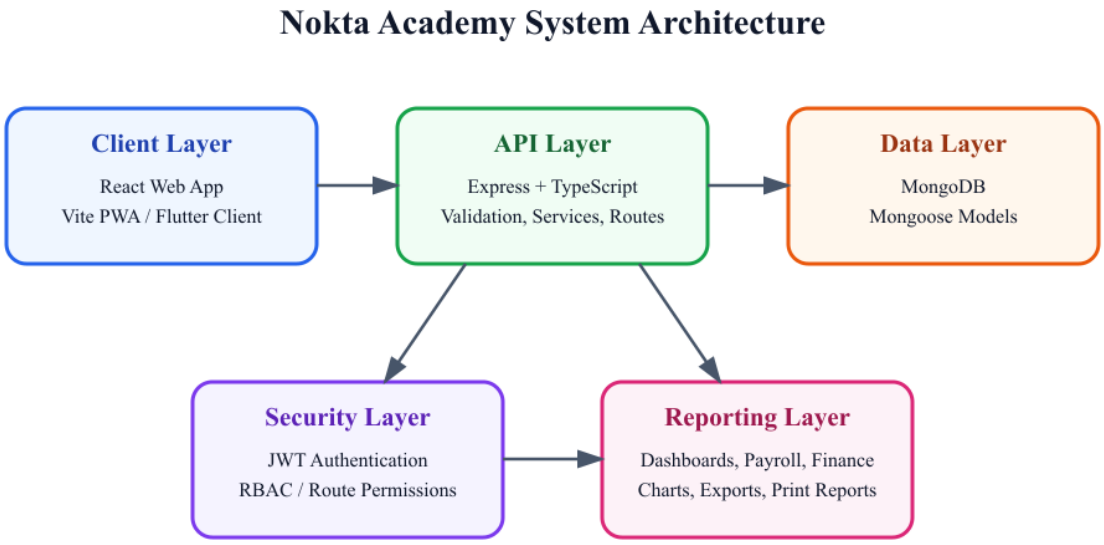
The figure shows that the methodology is not a single straight line. Requirements influence design, design guides implementation, testing produces feedback, and evaluation can create new refinements. This explains why the project could evolve from basic CRUD modules into

a more integrated academy management system with salary tax, financial reporting, and dashboard navigation.

3.12 Architectural Design

NAMS uses a layered web architecture. The presentation layer is built with React.js and Vite. The API layer uses Node.js and Express.js. The data layer uses MongoDB with Mongoose schemas. The security layer uses JWT authentication, RBAC, permission keys, and protected routes. The reporting layer includes dashboard analytics, finance aggregation, payroll reports, and academy financial reports. This architecture supports scalability and maintainability as recommended in software architecture literature (Bass et al., 2012).

Figure 3.11: Layered Architecture of NAMS



Main modules: Students, Teachers, Classes, Subjects, Attendance, Exams, Results, Payments, Payroll, Tax, Reports, Messages, Notification

The architecture diagram is shown after introducing the design layers. It visually explains how the client, API, database, security, and reporting layers communicate.

After examining the architecture, the reason for technology separation becomes clear. The frontend focuses on user interaction. The backend protects business logic and permissions. The database stores institutional records. The reporting layer transforms stored data into management information. This separation supports maintainability because each layer can be modified without rewriting the entire system.

Table 3.12: Technology Stack and Justification

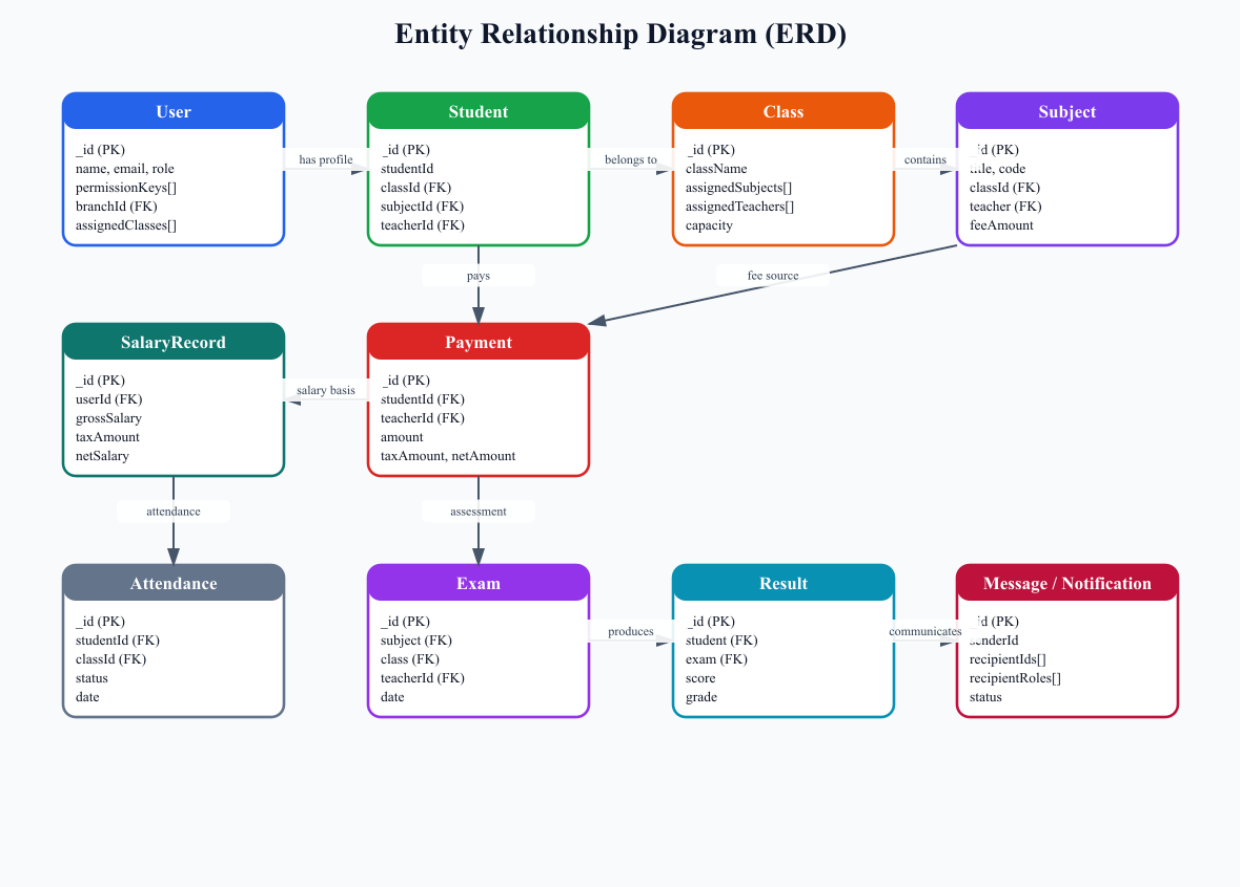
Layer	Technology	Version/Type	Justification
Frontend	React.js + Vite	React 18 / Vite 5	Responsive and component-based web UI
Backend	Node.js + Express.js	Node 22 / Express 4	Scalable API and service-based architecture
Database	MongoDB + Mongoose	MongoDB document DB	Flexible educational data modeling
Mobile	Flutter / mobile-ready frontend	Flutter-ready project structure	Cross-platform student/teacher access
Security	JWT + RBAC	Token and permission-based	Secure role-based access
Reporting	Dashboard + Recharts + backend aggregation	Charts and reports	Decision support and transparency
AI-readiness	Python/TensorFlow.js-ready design	Future module	Performance prediction and recommendations

The technology stack table follows the architecture figure and explains why each technology was selected.

3.13 Database Design

The database design is based on MongoDB collections represented conceptually through an ERD. Although MongoDB is a document database, the system uses references to preserve relationships among core entities. Student records reference class, subject, teacher, and family. Payment records reference student, teacher, class, subject, and salary records. SalaryRecord stores gross salary, tax, net salary, month, and payment status. This supports both operational transactions and management reports (Connolly, 2015).

Figure 3.13: Entity-Relationship Diagram



The ERD is placed after the database design explanation so that the reader can connect the text with the entities and relationships.

The ERD shows that the database is designed around relationships that matter to academy workflows. A student belongs to a class and subject, is assigned to a teacher, participates in attendance and exams, and creates fee payment records. Payments support financial reports and teacher salary. Salary records connect payroll and tax. This means the database is not only for storage; it is the foundation of operational automation.

Table 3.12: Core Relational/Document Schema

Collection	Key fields	Relationships
users	_id, name, email, role, branchId, salaryType, percentageRate	Teacher/admin/student accounts and role access
students	_id, studentId, classId, subjectId, teacherId, feeAmount, paidAmount	Links students to academic and financial records
classes	_id, className, assignedSubjects, assignedTeachers	Groups students and subjects
subjects	_id, title, classId, teacher, feeAmount	Connects teacher and class

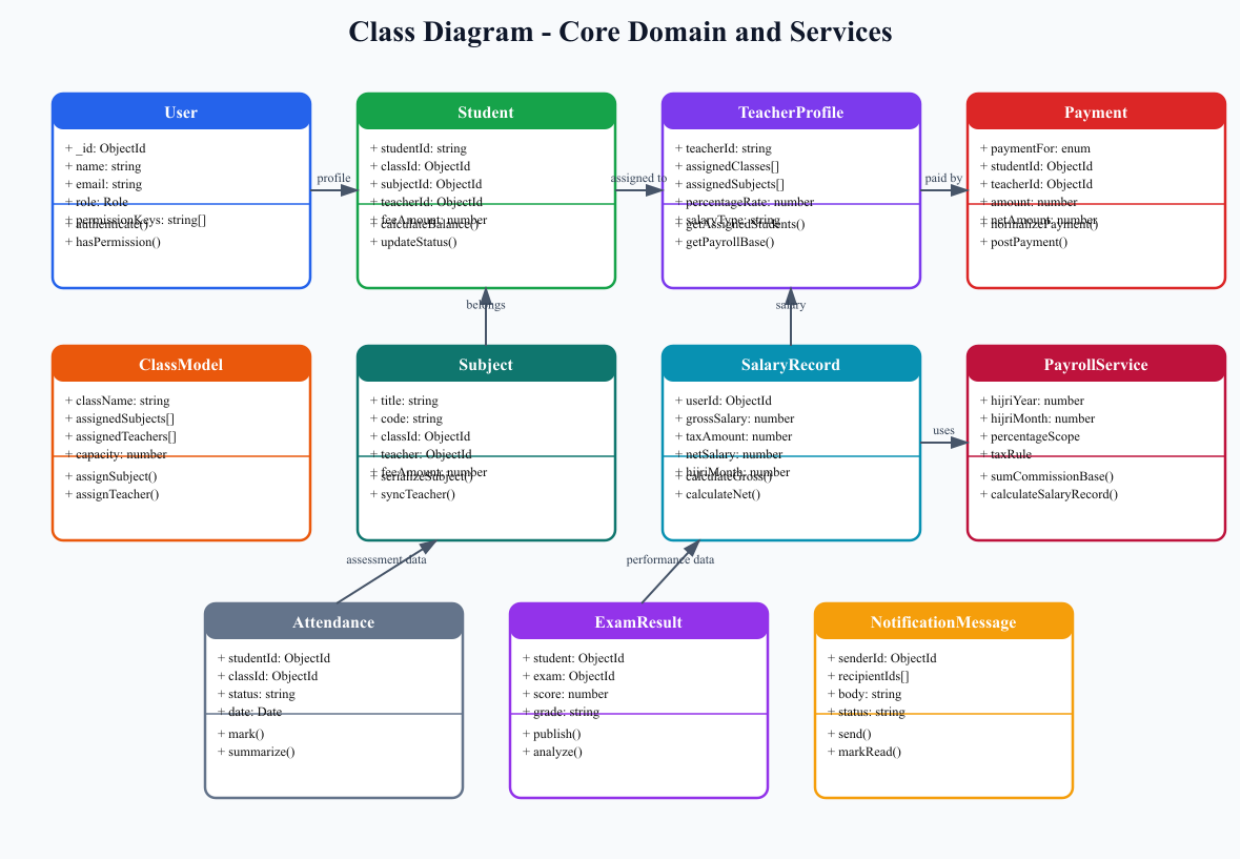
payments	_id, studentId, teacherId, amount, taxAmount, netAmount	Tracks student fees and salary payments
salary_records	_id, userId, grossSalary, taxAmount, netSalary	Stores payroll calculation result
attendance	_id, studentId, classId, status, date	Tracks attendance
results	_id, student, exam, score, grade	Stores academic performance

The schema table follows the ERD and converts graphical relationships into readable collection descriptions.

3.14 Class Diagram

The class diagram focuses on domain models and service relationships. It is useful because the ERD explains data relationships, while the class diagram explains how models and services behave. In NAMS, User, Student, TeacherProfile, Payment, SalaryRecord, PayrollService, Attendance, ExamResult, and NotificationMessage are central abstractions. Fowler (2004) and Larman (2004) support UML modeling as a way to communicate object-oriented structure and responsibilities.

Figure 3.14: Class Diagram



The class diagram appears after the explanation of why ERD alone is not enough for design. It shows attributes, behaviors, and service-level responsibilities.

After the class diagram, the system responsibilities become clearer. Models represent stored data, while services such as PayrollService represent business logic. This distinction is important for maintainability: salary calculation should not be scattered across UI pages; it should be handled by backend services that can be tested and reused.

3.15 Detailed Design

UI/UX Design

The UI/UX design focuses on simplicity, role-specific navigation, multilingual labels, light/dark mode readability, and dashboard-based access. Dashboard cards are clickable and route users to their related modules. This decision reduces navigation effort and supports perceived ease of use, which is important for user acceptance (Davis, 1989).

Algorithm Design

The most important algorithmic logic is teacher salary calculation. For each teacher, the system identifies assigned students, sums the student fee payments in the selected month, applies the teacher's commission percentage, calculates gross salary, applies government salary tax, and returns net payable salary.

Pseudocode: totalFee = sum(payments of assigned students); grossSalary = totalFee × commissionPercentage / 100; taxAmount = calculateAfghanistanSalaryTax(grossSalary); netSalary = grossSalary - taxAmount;

The complexity of this calculation depends mainly on the number of assigned students and payments in the period. With indexing on studentId, teacherId, and paymentDate, the query remains efficient for academy-scale data.

3.16 Security Design

Security is designed through JWT-based authentication, role-based access control, backend permission middleware, frontend protected routes, and validation of user input. Sensitive data includes student profiles, salary records, tax calculations, payment information, and user credentials. The system follows the principle of least privilege: each role sees only the sections required for its responsibility.

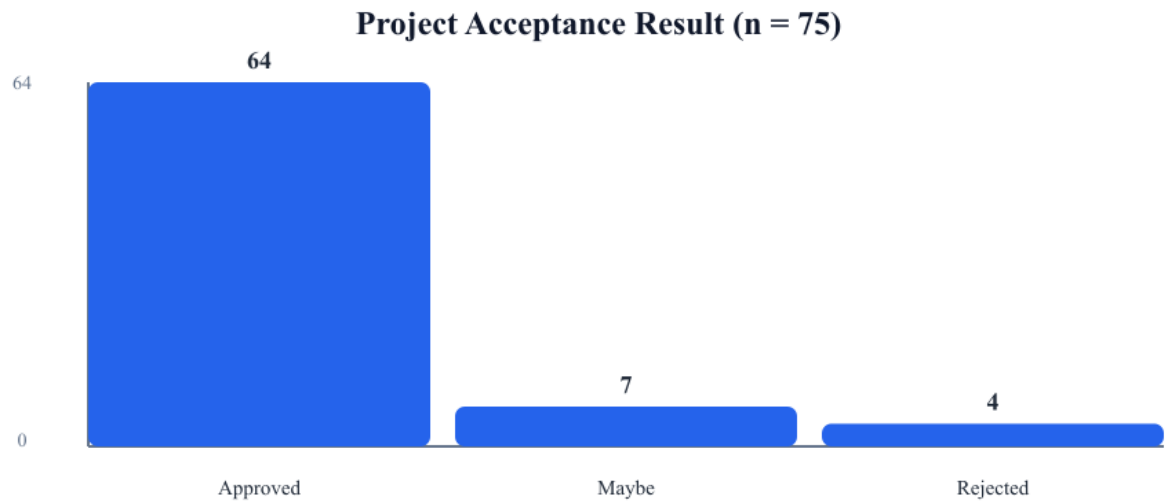
3.17 Testing Strategy

Testing includes unit tests, integration tests, system tests, and User Acceptance Testing. Unit tests focus on salary calculation, tax calculation, timetable validation, and permission policies. Integration tests focus on API workflows such as student creation, payment recording, salary calculation, and dashboard loading. UAT focuses on whether administrators, teachers, students, and parents can complete their expected workflows without confusion.

CHAPTER 4: Result

Before presenting the acceptance chart, it is necessary to recall the sampling structure. The population was 100 stakeholders and the analyzed sample was 75 respondents. The approval result therefore represents informed stakeholder feedback rather than a random public survey. This context is important for interpreting the chart.

Figure 4.1: Project Acceptance Result

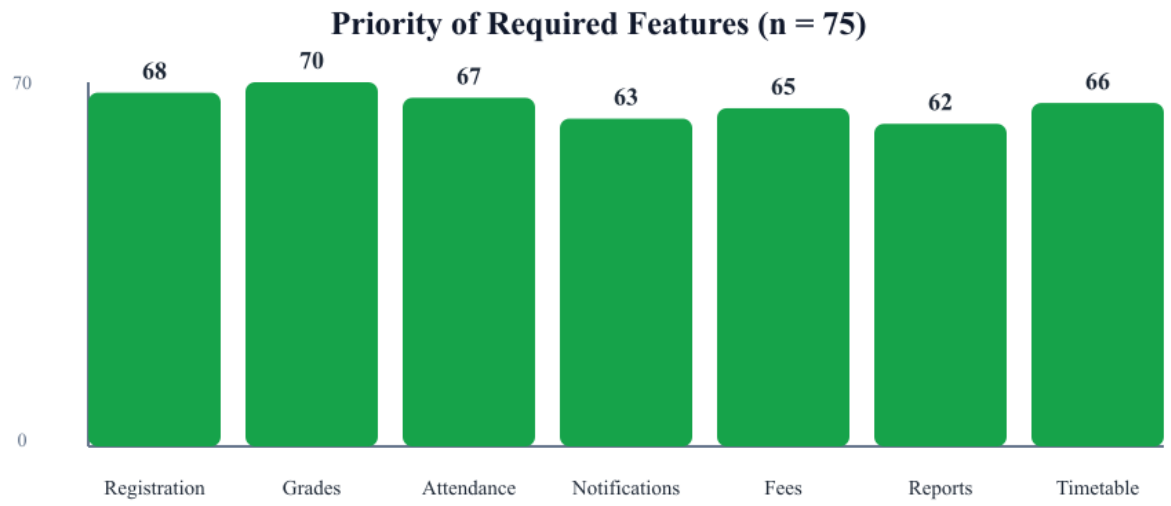


The chart shows that 64 of 75 respondents approved the project, producing an 85.33% approval rate.

The chart confirms that the majority of respondents support the project. This result answers the practical part of the main research question: stakeholders believe that an integrated online system can improve academy operations. The remaining “maybe” and “rejected” responses also suggest that future deployment should include user training and gradual adoption.

Before the feature chart, it is important to identify why feature priority matters. If the most requested features are not implemented, the system may be technically complete but socially unacceptable. Therefore, feature priority results were compared with the modules actually implemented in the project.

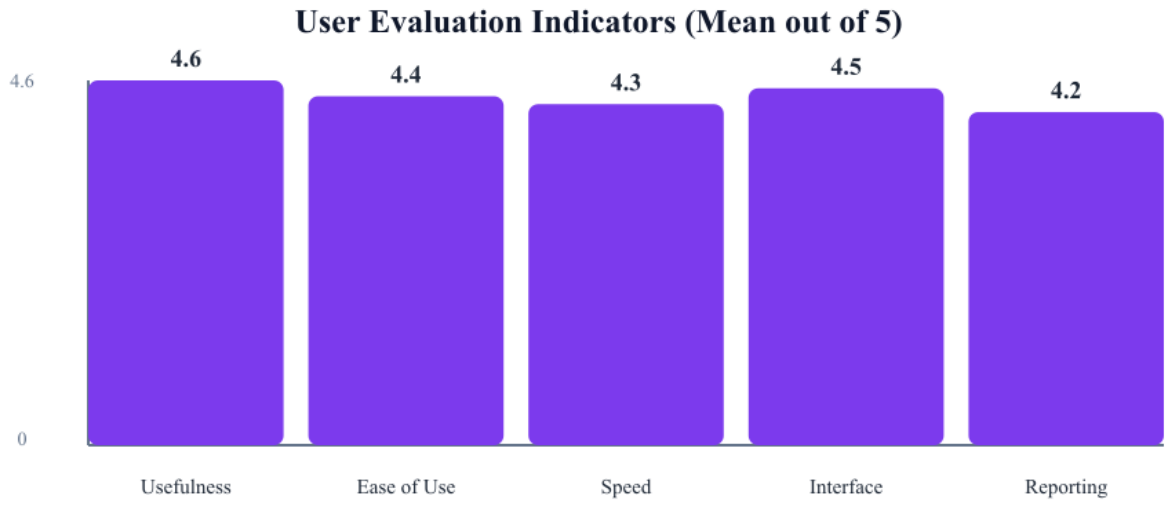
Figure 4.2: Feature Priority Chart



The chart summarizes the major features selected by respondents.

The feature priority chart shows high demand for grade management, online registration, attendance, timetable, fee payment, notifications, and reports. These results align directly with NAMS modules. The alignment strengthens the argument that implementation decisions were based on user needs rather than developer preference.

Figure 4.3: User Evaluation Indicators



This chart summarizes perceived usefulness, ease of use, speed, interface, and reporting value.

The user evaluation chart complements the acceptance result. It suggests that users are not only approving the project in general, but also rating its practical qualities positively. This relates to TAM because usefulness and ease of use are both central to adoption (Davis, 1989).

Table 4.1: Research Question Answer Matrix

Research Question	Answer found in project
How can the implementation of an integrated online academy management system enhance operational efficiency and personalized learning?	By centralizing academic, financial, and communication workflows and preparing structured data for analytics.
What current inefficiencies can be addressed through digitization?	Manual registration, scattered payments, delayed reports, weak communication, and unclear salary/tax processes.
How can AI-driven analytics improve student learning outcomes and academic monitoring?	By using structured attendance, result, timetable, and student records for future risk detection and recommendation.
What are best practices for communication and financial transparency?	Integrated messaging, notifications, fee payment tracking, payroll, tax calculation, and financial reports.
How can Node.js, MongoDB, React, and Flutter support scalable and maintainable educational platforms?	Through separated frontend, API, database, mobile-ready access, modular routes, and reusable components.

The table directly answers every research question from the proposal.

CHAPTER 5: CONCLUSION

The Nokta Academy Management System successfully converts the proposal idea into a working software engineering project. It integrates academic, administrative, financial, communication, and reporting functions in a centralized platform. The project aligns with the proposal's objectives by using Node.js, Express.js, MongoDB, React.js, and mobile-ready design principles, while also preparing for AI-based analytics.

The system improves operational efficiency by reducing manual work, linking data across modules, and producing management reports. It improves financial transparency by connecting student fee payments with salary calculation, tax deduction, and academy financial reporting. It improves communication through notifications and messaging. It improves future personalization potential by storing structured academic and attendance data that can later support AI analytics.

Future work should complete the AI performance prediction module, conduct longer-term user evaluation, deploy the system on secure cloud infrastructure, strengthen backup and disaster recovery, and expand the Flutter mobile application for daily student and teacher use.

APPENDIX A: TECHNICAL ARTIFACTS

Some technical details are placed in the appendix because they are important for project completeness but would interrupt the main academic discussion if included in the body chapters. These include API modules, core algorithms, testing scripts, and implementation evidence.

Appendix Table A.1: Technical Artifacts Not Fully Listed in Main Chapters

Artifact	Description
Backend modules	auth, users, students, teachers, classes, subjects, attendance, exams, results, payments, payroll, finance, reports
Frontend modules	dashboard, resources, payments, finance, payroll, reports, messages, notifications
Core algorithm	Teacher salary calculation from assigned student payments and commission percentage
Security artifacts	JWT authentication, route permission middleware, protected frontend routes

Testing scripts	salary tax, payroll commission, route policy, timetable validation, HTTP errors

The appendix supports completeness without overloading the main narrative.

REFERENCES

Aldowah, H., Al-Samarraie, H., & Fauzy, W. M. (2019). Educational data mining and learning analytics for 21st century higher education: A review and synthesis. Telematics and Informatics, 37, 13–49. https://doi.org/10.1016/j.tele.2019.01.007

Al-Samarraie, H., & Saeed, N. (2022). Understanding the adoption of smart educational systems: A systematic literature review. Education and Information Technologies, 27(1), 611–639. https://doi.org/10.1007/s10639-021-10711-3

American Psychological Association. (2020). Publication manual of the American Psychological Association (7th ed.). American Psychological Association. https://doi.org/10.1037/0000165-000

Bass, L., Clements, P., & Kazman, R. (2012). Software architecture in practice (3rd ed.). Addison-Wesley.

Brooke, J. (1996). SUS: A quick and dirty usability scale. In P. W. Jordan, B. Thomas, B. A. Weerdmeester, & I. L. McClelland (Eds.), Usability evaluation in industry (pp. 189–194). Taylor & Francis.

Connolly, T. M., & Begg, C. E. (2015). Database systems: A practical approach to design, implementation, and management (6th ed.). Pearson.

Creswell, J. W., & Creswell, J. D. (2018). Research design: Qualitative, quantitative, and mixed methods approaches (5th ed.). SAGE Publications.

Davis, F. D. (1989). Perceived usefulness, perceived ease of use, and user acceptance of information technology. MIS Quarterly, 13(3), 319–340. https://doi.org/10.2307/249008

Delen, D. (2010). A comparative analysis of machine learning techniques for student retention management. Decision Support Systems, 49(4), 498–506. https://doi.org/10.1016/j.dss.2010.06.003

DeLone, W. H., & McLean, E. R. (2003). The DeLone and McLean model of information systems success: A ten-year update. Journal of Management Information Systems, 19(4), 9–30. https://doi.org/10.1080/07421222.2003.11045748

Dikert, K., Paasivaara, M., & Lassenius, C. (2016). Challenges and success factors for large-scale agile transformations: A systematic literature review. Journal of Systems and Software, 119, 87–108. https://doi.org/10.1016/j.jss.2016.06.013

Fowler, M. (2004). UML distilled: A brief guide to the standard object modeling language (3rd ed.). Addison-Wesley.

Garrett, J. J. (2011). The elements of user experience: User-centered design for the web and beyond (2nd ed.). New Riders.

Hevner, A. R., March, S. T., Park, J., & Ram, S. (2004). Design science in information systems research. MIS Quarterly, 28(1), 75–105.

Ifenthaler, D., & Yau, J. Y.-K. (2020). Utilising learning analytics for study success: Reflections on current empirical findings. Research and Practice in Technology Enhanced Learning, 15, Article 17. https://doi.org/10.1186/s41039-020-00138-1

International Organization for Standardization, & International Electrotechnical Commission. (2011). Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models (ISO/IEC 25010:2011).

International Organization for Standardization, International Electrotechnical Commission, & Institute of Electrical and Electronics Engineers. (2018). Systems and software engineering — Life cycle processes — Requirements engineering (ISO/IEC/IEEE 29148:2018).

Larman, C. (2004). Applying UML and patterns: An introduction to object-oriented analysis and design and iterative development (3rd ed.). Prentice Hall.

L ikert, R. (1932). A technique for the measurement of attitudes. Archives of Psychology, 22(140), 1–55.

Nielsen, J. (1994). Usability engineering. Morgan Kaufmann.

Pressman, R. S., & Maxim, B. R. (2020). Software engineering: A practitioner's approach (9th ed.). McGraw-Hill Education.

Romero, C., & Ventura, S. (2020). Educational data mining and learning analytics: An updated survey. WIREs Data Mining and Knowledge Discovery, 10(3), e1355. https://doi.org/10.1002/widm.1355

Siemens, G. (2013). Learning analytics: The emergence of a discipline. American Behavioral Scientist, 57(10), 1380–1400. https://doi.org/10.1177/0002764213498851

Sommerville, I. (2016). Software engineering (10th ed.). Pearson.

Venkatesh, V., Morris, M. G., Davis, G. B., & Davis, F. D. (2003). User acceptance of information technology: Toward a unified view. MIS Quarterly, 27(3), 425–478.

Wieringa, R. J. (2014). Design science methodology for information systems and software engineering. Springer.